

Outcome Prediction Quiz: Hyperparameter Scaling

CS 312 · 140 points

Answer sheet. Write your Name and SUNet ID, mark every answer and write your rationale on the separate answer sheet at the end of this booklet (pages S1–S2). Only the answer sheet is graded; the option lists in the questions are for reference.

Instructions. Predict the outcomes of the following experiments. Each question has two parts:

- **From the assignment** restates, for reference, the assignment setting the question starts from. Nothing in it is new.
- **This question changes** lists every difference from that setup. Everything not listed there is unchanged.

Check one box in every row of the answer sheet. **Scoring:** the correct option earns full points, and an adjacent interval earns half. Each question has three rationale fields: the assignment problem you cite, what that experiment showed (with numbers), and why it implies your answer. Detailed proper citation, with experimental details and sound rationale can return 75% of missed points on a question. Appendix A prints the complete code of every change.

Default setup. The standard setup of the Hyperparameter Scaling assignment: Llama-style architecture with 8 layers, hidden size 512, 8 attention heads, QK-norm, unscaled RoPE, a 4k vocab size tokenizer, and about 35M parameters (d8); fresh data (no epoching) with the token budget stated in each question; AdamW with $\beta_1, \beta_2 = (0.9, 0.95)$ and $\epsilon = 10^{-8}$, linear decay learning rate schedule with 1% linear warmup, peak learning rate 0.003, context length 1024, batch size 64, weight decay 0.1 (masked), and 1.0 grad clipping; model and data seed 42. Lower validation loss is better.

Conventions. We follow the same convention as the rest of A2

- **Fitted optimum** of a loss–LR curve: the vertex of the quadratic in $\log(\text{LR})$ through the best sampled LR and its two grid neighbours. The **fitted minimum loss** is the quadratic’s value at that vertex.
- **Transfer penalty:** the loss at a transferred LR minus the fitted minimum loss of the target curve.

Prediction

Q1. Depth scaling with a shared block

Difficulty: Hard

20 points

From the assignment (reference only). Recall the assignment's five-step depth stress test, in which we compared depth-scaling variants of μP . Width was $n = n_0 = 64$, one attention head of dimension 64, reference depth 2, depth ratio $r = L/2$. To obtain better scaling, each attention and feedforward branch is multiplied by c_L before it is added to the residual stream. With base LR η , the three prescriptions set the following for the blocks (embedding, readout and final normalization keep LR η and $\epsilon = 10^{-8}$):

Prescription	Residual c_L	Block LR	Block Adam ϵ
A. Standard μP	1	η	10^{-8}
B. Depth- μP	$r^{-1/2}$	$\eta r^{-1/2}$	$10^{-8} r^{-1/2}$
C. CompleteP	r^{-1}	η	$10^{-8} r^{-1}$

This question changes. Depth $L = 1000$, so $r = 500$ and c_L is 1, .0447 and .002 for A, B and C. The 1,000 blocks are **one shared block**: a single block, initialized like one block of the stress test, applied 1,000 times with shared weights. It takes its prescription's block LR and ϵ .

(a) Residual stream at initialization (12 points, 4 per row). Before any update, take the RMS of the final residual stream (the input to the final RMSNorm, pooled over the 8 fixed probe sequences). Let

$$\rho = \frac{\text{this RMS in the shared-block model}}{\text{this RMS in the stress-test model with 1,000 independent blocks'}}$$

with the same prescription and seed. Which interval contains ρ ?

Prescription	Option 1	Option 2	Option 3	Option 4
A. Standard μP	$\cdot (0, 10)$	$\cdot [10, 100)$	$\cdot [100, 1000)$	$\cdot [1000, \infty)$
B. Depth- μP	$\cdot (0, 10)$	$\cdot [10, 100)$	$\cdot [100, 1000)$	$\cdot [1000, \infty)$
C. CompleteP	$\cdot (0, 10)$	$\cdot [10, 100)$	$\cdot [100, 1000)$	$\cdot [1000, \infty)$

(b) Loss after five updates (8 points). For each prescription, take the shared-block model's lowest validation loss after five updates over base LR's $\{.003, .01, .03\}$. Which prescription attains the lowest of these three losses?

- 1. A. Standard μP · 2. B. Depth- μP · 3. C. CompleteP · 4. All three within .005

Mark your answers and write your rationale on answer sheet page S1.

Q2. Predicting loss under joint LR and WD changes

Difficulty: Medium

20 points

From the assignment (reference only). Recall the experiments that swept learning rate and weight decay jointly. We keep the token budget and the LR–WD grid the same as in the assignment.

This question changes. The LR schedule is **cosine decay to zero** instead of linear decay (same 1% warmup), at **1.2288B tokens**, on the same 3×3 LR–WD grid. With cosine decay, the grid's best cell is peak LR .003, WD .2 ($\eta\lambda = 6 \times 10^{-4}$), with loss 2.8354. Configurations A–F are six other cells of this grid:

Configuration	Peak LR η	WD λ	$\eta\lambda (\times 10^{-4})$
A	.006	.1	6
B	.0015	.4	6
C	.006	.4	24
D	.0015	.1	1.5
E	.006	.2	12
F	.003	.4	12

Let $L_A, \dots, L_F$ denote the losses of configurations A–F.

(a) Which loss ranking do you predict? (10 points) Choose the option closest to the measured ranking (fewest pairs in the wrong order).

- 1. $L_A < L_B < L_F < L_E < L_D < L_C$
- 2. $L_D < L_B < L_F < L_E < L_A < L_C$
- 3. $L_F < L_E < L_A < L_B < L_C < L_D$
- 4. $L_C < L_E < L_F < L_A < L_B < L_D$

(b) (10 points) Which interval contains configuration C's excess loss, $L_C - 2.8354$?

- 1. $[0, .005)$
- 2. $[.005, .015)$
- 3. $[.015, .025)$
- 4. $[.025, \infty)$

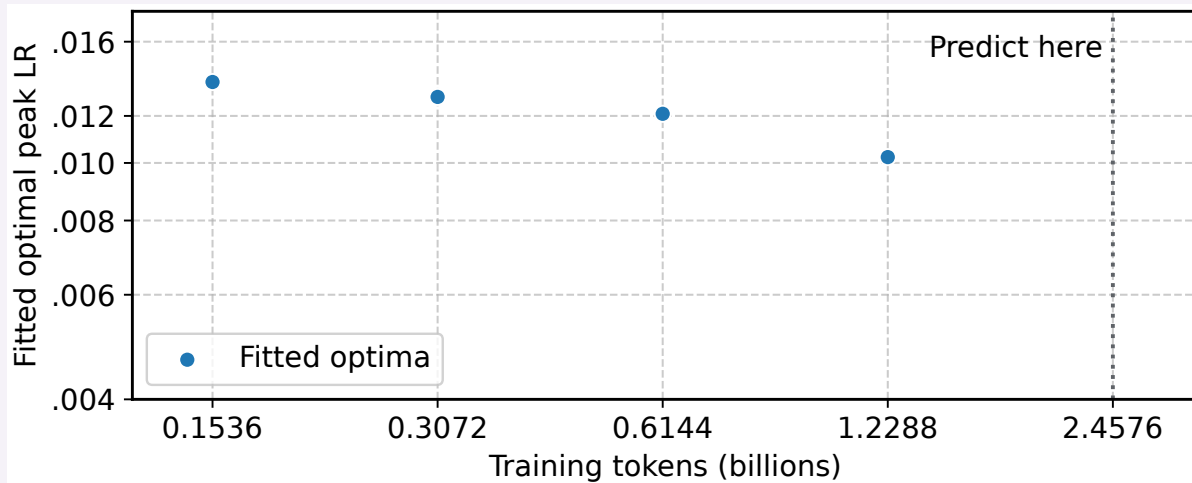
Mark your answers and write your rationale on answer sheet page S1.

Q3. Extrapolating Hyperball's optimal LR

Difficulty: Easy

20 points

From the assignment (reference only). Recall our study of Hyperball. We ran scaling ladders, with Hyperball for every linear-layer weight matrix (including the readout), and Adam on embeddings and normalization gains at peak LR $.1041 \eta$, where η is the Hyperball peak LR; for this problem, we disabled weight decay. For reference, we provide the fitted optima of three supplied budgets and of the 1.2288B target on log-log axes.



This question changes. The token budget is **2.4576B tokens** (the dotted line), with peak LR $\{.006, .01, .015\}$.

Which interval contains the fitted optimum at 2.4576B tokens? (20 points)

- 1. $[\cdot0075, \infty)$ · 2. $[\cdot0055, \cdot0075)$ · 3. $[\cdot004, \cdot0055)$ · 4. $(0, \cdot004)$

Mark your answers and write your rationale on answer sheet page S1.

Q4. Batch size and momentum

Difficulty: Easy

20 points

From the assignment (reference only). Recall that we ran experiments varying momentum at 614.4M tokens. We build upon that set of experiments, keeping $\beta_2 = 0.95$ fixed, and varying only β_1 .

This question changes. We only consider two different batch size / learning rate / weight decay combinations, and β_1 takes three values: **A:** $\beta_1 = 0$, **B:** $\beta_1 = .9$, **C:** $\beta_1 = .98$.

Batch size B	Peak LR	WD	Loss at $\beta_1 = .9$
8	.00075	.1	2.936603
256	.0015	1.944	2.968249

At each batch size, let L_A, L_B, L_C be the losses with $\beta_1 = 0, .9, .98$.

(a) Rankings (5 points per batch size).

Option 1: $L_C < L_B < L_A$ **Option 2:** $L_A < L_B < L_C$
Option 3: $L_A < L_C < L_B$ **Option 4:** $L_B < L_C < L_A$

Choose one option for $B = 8$ and one for $B = 256$.

(b) Cost of removing momentum (5 points per batch size). Which interval contains $L_A - L_B$?

B	Option 1	Option 2	Option 3	Option 4
8	$\cdot (-\infty, .01)$	$\cdot [.01, .06)$	$\cdot [.06, .15)$	$\cdot [.15, \infty)$
256	$\cdot (-\infty, .01)$	$\cdot [.01, .06)$	$\cdot [.06, .15)$	$\cdot [.15, \infty)$

Mark your answers and write your rationale on answer sheet page S2.

Q5. Does learning-rate transfer survive changing the readout?

Difficulty: Medium

20 points

From the assignment (reference only). Recall that we studied width scaling at 153.6M tokens, with hidden width n ($n/64$ heads), $n_0 = 512$, $m = n/n_0$ and peak base LR η . G is one fixed matrix of standard Gaussian draws (entries truncated at ± 3), shared by the embedding and the readout; “ G/n ” means those draws divided by n , so its entries have standard deviation about $1/n$. For reference, here are the scaling rules for both the course baseline and μP .

Setting	Course baseline	μP
Embedding initialization	G/n	G/n_0
Readout initialization	$G/\sqrt{n}$	$G/\sqrt{n_0}$
Readout forward multiplier	1	$1/m$
Hidden-matrix peak LR	η	η/m

Embedding, readout and normalization parameters keep peak LR η . At width 512 every prescription (including almost- μP below) is the default d8 model.

This question changes. A third prescription, **almost- μP** : μP 's embedding initialization (G/n_0) and hidden-matrix LR (η/m), but the course baseline's readout initialization ($G/\sqrt{n}$) and readout multiplier (1). It is trained at width 1024 ($m = 2$) with peak LR's $\{.00075, .0015, .003, .006, .012\}$.

(a) (10 points) Let $\eta^*(1024)$ be almost- μP 's fitted optimum at width 1024 and $\eta^*(512) = .002378$. Which interval contains $\eta^*(1024)/\eta^*(512)$?

- 1. $(0, .5)$ · 2. $[.5, .8)$ · 3. $[.8, 1.25)$ · 4. $[1.25, \infty)$

(b) (10 points) At width 1024, which interval contains almost- μP 's transfer penalty for the transferred peak LR .003?

- 1. $[.15, \infty)$ · 2. $[.06, .15)$ · 3. $[.01, .06)$ · 4. $[0, .01)$

Mark your answers and write your rationale on answer sheet page S2.

Q6. Hyperball across widths

Difficulty: Hard

20 points

From the assignment (reference only).

- **Hyperball:** Recall that in Hyperball, every linear-layer weight matrix, including the readout, takes an Adam-direction step and is projected back to its initial Frobenius norm; embeddings and normalization gains use Adam at peak LR $.1041 \eta$; for this problem, we disabled weight decay.
- **Course-baseline width sweep:** From our assignment, we did width sweeps at 153.6M tokens with $n/64$ heads and feedforward width $3.5n$; the default initialization (embedding G/n , readout $G/\sqrt{n}$, hidden-matrix variance $1/k$ for fan-in k); readout multiplier 1. In this problem we consider scaling in **the course baseline parametrization**, not μP . At $n = 512$ this is the default d8 model.

This question changes. The assignment's width sweep is trained with **Hyperball** instead of AdamW, at widths $n \in \{128, 256, 512, 1024\}$ and peak LR $\eta \in \{.006, .01, .015, .022, .03, .045\}$.

(a) (10 points) Fit $\eta^*(n) = c n^p$ to the fitted optima at widths 128, 256, 512 and 1024. Which interval contains p ?

- 1. $(-\infty, -.75)$ · 2. $[-.75, -.25)$ · 3. $[-.25, .25)$ · 4. $[.25, \infty)$

(b) (10 points) At width 1024, which interval contains the transfer penalty of width 512's best sampled LR?

- 1. $[0, .005)$ · 2. $[.005, .02)$ · 3. $[.02, .08)$ · 4. $[.08, \infty)$

Mark your answers and write your rationale on answer sheet page S2.

Q7. Scaling LR and WD to batch size 512

Difficulty: Medium

20 points

From the assignment (reference only). Recall our investigations into batch size, weight decay, and learning rate dependence. We start with the default setup of $B = 64$ at 614.4M tokens, and for your reference the best configuration in this case has peak LR .003, WD .2 ($\eta\lambda = 6 \times 10^{-4}$), with loss 2.9178.

This question changes. Batch size $B = 512$, so training takes eight times fewer updates, with peak LR and WD set by one of five rules:

Recipe	Rule	Peak LR η	WD λ	$\eta\lambda (\times 10^{-4})$
A	Keep both	.003	.2	6
B	$\text{LR} \times \sqrt{8}$	.00849	.2	17
C	$\text{LR} \times 8$	.024	.2	48
D	$\text{WD} \times 8$	.003	1.6	48
E	$\text{LR} \times \sqrt{8}$ and $\text{WD} \times \sqrt{8}$	.00849	.566	48

Let $L_A, \dots, L_E$ denote the losses of recipes A–E.

(a) (10 points) Which recipe gives the lowest loss?

☐ A ☐ B ☐ C ☐ D ☐ E

(b) (10 points) Which interval contains $L_A - \min(L_A, \dots, L_E)$, the cost of keeping the batch-64 pair?

☐ 1. $[0, .02)$ ☐ 2. $[.02, .05)$ ☐ 3. $[.05, .15)$ ☐ 4. $[.15, \infty)$

Mark your answers and write your rationale on answer sheet page S2.

Appendix: the quiz code *Complete; nothing else changes*

This appendix prints all of the quiz's code. It builds on the released Assignment 2 starter you used for the assignment: one change to the stress scaffold (A.1) and three new files (A.2, A.3). Nothing else in the starter changes. The code uses these starter names:

- `config`: a training run with the Default setup at 153.6M tokens, changed only by its keyword arguments. `optimizer_name='adamh'` selects the starter's Hyperball, with no weight decay.
- `AutoregressiveLM` and `initialize_model`: the course model and its default initialization (embedding G/n , readout $G/\sqrt{n}$, hidden matrices with variance $1/k$ for fan-in k). The model's output is `lm_head` applied to the final hidden state.
- `LMConfig`: the model shape, given as name, vocabulary, context, width, feedforward width, layers, heads and key-value heads.
- `should_apply_weight_decay`: the course weight-decay mask (hidden and readout matrices decay; embeddings and gains do not). `ADAMW_EPSILON` is 10^{-8} .
- `stress.run` and `StressConfig`: the assignment's five-step stress test.

Each stress cell `cell` in `runs.py` (A.3) is run as

```
init, groups = depth_prescription(cell['method'])
stress.run(StressConfig(head_dim=64, **cell['config']), train, val, base_lr=cell['lr'],
           initialize_fn=init, groups_fn=groups, alignment=False)
```

where `train` and `val` are the scaffold's five fixed training batches and 64 validation sequences; a cell that diverges counts as infinite loss. Each training run trains on its configuration, and its outcome is the final validation loss. Every graded run uses seed 42. No experiments need to be run for this quiz.

A.1 — The stress scaffold: one block applied at every depth (Q1)

A diff of the starter file: red lines are removed, green lines are added.

```
--- a/experiments/a2/stress.py
+++ b/experiments/a2/stress.py
@@ -37,6 +37,7 @@ class StressConfig:
     checkpoint_blocks: int = 16
     seed: int = 42
     precision: str = 'fp32'
+    tied: bool = False # Quiz 2 Q1: one block module applied at every depth

    def validate(self):
        for key, value in asdict(self).items():
@@ -44,6 +45,8 @@ class StressConfig:
        if value not in ('fp32', 'mp'):
            raise ValueError('precision must be fp32 (width) or mp (depth)')
        continue
+
+    if key == 'tied':
+        continue
        if not isinstance(value, int) or isinstance(value, bool):
            raise ValueError(f'{key} must be an integer')
        if key != 'seed' and value <= 0:
@@ -91,7 +94,8 @@ class StressTransformer(nn.Module):
    # No implicit PyTorch initialization serves as an assignment answer.
    with torch.device('meta'):
        self.embed = nn.Embedding(c.vocab, c.width)
-        self.blocks = nn.ModuleList([GatedSiLUBlock(c) for _ in range(c.depth)])
+        blocks = [GatedSiLUBlock(c) for _ in range(1 if c.tied else c.depth)]
+        self.blocks = nn.ModuleList(blocks * c.depth if c.tied else blocks)
        self.norm = LlamaRMSNorm(c.width, 1e-5)
        self.head = nn.Linear(c.width, c.vocab, bias=False)
    self.to(dtype=torch.float32)
```

```

@@ -170,7 +174,7 @@ class StressProbe:
    self.maps = {name: module for name, module in model.named_modules()
                  if isinstance(module, nn.Linear)} if alignment else {}
    n = len(model.blocks)
-    self.positions = sorted({0, n // 2, n - 1})
+    self.positions = [] if model.config.tied else sorted({0, n // 2, n - 1})

    @torch.no_grad()
    def sample(self):

```

With tied=True the same block object sits at every depth. Autograd sums its gradient over all 1,000 applications and parameters() lists it once, so Q1's prescription (A.2) gives it the shared-block LR and ϵ . The probe records no per-block positions for a shared block.

A.2 — New file: the prescriptions (Q1, Q5, Q6)

depth_scales is the Q1 table. AlmostMuPLM keeps the starter's forward pass, so its readout multiplier is 1.

```

experiments/quiz2/prescriptions.py
"""The prescriptions Quiz 2 trains, written from the question tables so that the quiz needs
only the released starter: Q1's three depth prescriptions for the stress scaffold, Q5's
almost-muP, and the width-n course model of Q5 and Q6."""
import torch

from model_config import LMConfig
from modeling import AutoregressiveLM, initialize_model
from optimizers import ADAMW_EPSILON, should_apply_weight_decay

# Q1: the stress scaffold at width n = n0 = 64, reference depth 2, r = L/2.
HIDDEN = ('q', 'k', 'v', 'o', 'gate', 'up', 'down')
EPS = 1e-8

def kind(name):
    """'hidden', 'block_norm', or 'outer' (embedding, readout, final norm)."""
    parts = name.split('.')
    if parts[0] != 'blocks':
        return 'outer'
    return 'hidden' if parts[2] in HIDDEN else 'block_norm'

def depth_scales(method, depth, depth0=2):
    """The Q1 table: residual multiplier c_L, block LR factor and block Adam-eps factor."""
    r = depth / depth0
    if method == 'mup':
        return 1., 1., 1.
    if method == 'depth_mup':
        return r ** -.5, r ** -.5, r ** -.5
    if method == 'completep':
        return r ** -1., 1., r ** -1.
    raise ValueError(method)

def depth_prescription(method):
    """(initialize, parameter_groups) for stress.run. At n = n0 the width rules are the identity:
    embedding variance 1, hidden variance 1/fan-in, readout variance 1/n, gains 1, readout
    multiplier 1. Embedding, readout and final norm keep LR eta and eps 1e-8."""
    def initialize(model):
        n = model.config.width
        if n != 64:
            raise ValueError('Q1 runs at width n0 = 64')
        for name, p in model.named_parameters(): # a shared block is listed once
            if p.ndim == 1:
                p.fill_(1.)
            elif name == 'embed.weight':
                p.normal_(0., 1.)
            elif name == 'head.weight':
                p.normal_(0., n ** -.5)

```

```

        else:
            p.normal_(0., p.shape[1] ** -.5)
        model.output_multiplier = 1.
        c, _, _ = depth_scales(method, model.config.depth)
        for b in model.blocks:
            b.residual_multiplier = c

    def parameter_groups(model, base_lr):
        _, block_lr, block_eps = depth_scales(method, model.config.depth)
        return [dict(params=p, lr=base_lr, eps=EPS) if kind(name) == 'outer' else
                dict(params=p, lr=base_lr * block_lr, eps=EPS * block_eps)
                for name, p in model.named_parameters()]
    return initialize, parameter_groups

# Q5 and Q6: the course model at hidden width n, with m = n/512.
def width_model_config(width):
    """n/64 heads of dimension 64 and feedforward width 3.5n; width 512 is the default d8."""
    if width % 64:
        raise ValueError('width must be a multiple of the 64-dimensional head')
    return LMConfig(f'a2-d8-w{width}', 4096, 1024, width, int(3.5 * width), 8,
                    width // 64, width // 64)

class AlmostMuPLM(AutoregressiveLM):
    """Q5's almost-muP: muP's embedding initialization G/n0 and hidden-matrix LR eta/m (see
    build_optimizer), with the course baseline's readout initialization G/sqrt(n) and readout
    multiplier 1."""
    def __init__(self, config, *, base_width=512, **kwargs):
        super().__init__(config, **kwargs)
        self.width_ratio = config.hidden_size / base_width

    def initialize_parameters(self):
        initialize_model(self) # course baseline: embedding G/n, readout G/sqrt(n)
        with torch.no_grad():
            self.model.embed_tokens.weight.mul_(self.width_ratio) # G/n -> G/n0

def build_optimizer(model, optimizer_name, learning_rate, weight_decay, beta1, beta2):
    """Fused AdamW (eps 1e-8): hidden matrices at LR eta/m; embedding, readout and gains at eta;
    the course weight-decay mask."""
    if optimizer_name != 'adamw':
        raise ValueError('almost-muP uses AdamW')
    hidden_scale = 1 / model.width_ratio
    special = {id(model.lm_head.weight), id(model.model.embed_tokens.weight)}
    groups = {}
    seen = set()
    for module in model.modules():
        for name, p in module.named_parameters(recurse=False):
            if id(p) in seen or not p.requires_grad:
                continue
            seen.add(id(p))
            hidden = isinstance(module, torch.nn.Linear) and id(p) not in special
            lr = learning_rate * (hidden_scale if hidden else 1.)
            wd = weight_decay if should_apply_weight_decay(module, name) else 0.
            groups.setdefault((lr, wd), []).append(p)
    fused = any(p.is_cuda for ps in groups.values() for p in ps)
    return torch.optim.AdamW([dict(params=ps, lr=lr, weight_decay=wd) for (lr, wd), ps in groups.items()],
                              lr=learning_rate, betas=(beta1, beta2), eps=ADAMW_EPSILON,
                              weight_decay=0., fused=fused)

```

A.3 — New files: every graded run (Q1–Q7)

runs.py lists the stress cells of Q1 and the training runs of Q2–Q7 (each marked by a comment).

```

experiments/quiz2/__init__.py
"""Quiz 2: the prescriptions its questions train and every graded run."""

```

```

experiments/quiz2/runs.py
"""Every graded Quiz 2 run (seed 42). Stress cells call stress.run with Q1's depth
prescriptions (prescriptions.py); training runs use the A2 config factory (unscaled RoPE,
seed-42 global-shuffle prefix, fused AdamW, 1% warmup + linear decay, batch 64, clipping 1,
WD .1 unless stated)."""
from experiments.a2.modal_launcher import config
from experiments.quiz2.prescriptions import width_model_config

SQRT8 = 8 ** .5
# Q7: batch 512 recipes from the B = 64 reference pair (LR .003, WD .2).
Q7_RECIPES = dict(A=(.003, .2), B=(.003 * SQRT8, .2), C=(.024, .2), D=(.003, 1.6),
                  E=(.003 * SQRT8, .2 * SQRT8))

def stress_name(policy, width, depth, lr, seed, tied):
    return f"{policy}-n{width}-L{depth}{'-tied' if tied else ''}-lr{lr:g}-s{seed}"

def stress_cells(seed=42):
    cells = []
    for method in ('mup', 'depth_mup', 'completep'): # Q1: width 64, depth 1000, mixed precision
        for tied, lrs in ((True, (.003, .01, .03)), (False, (.01, ))): # untied: initial state only
            for lr in lrs:
                cells.append(dict(name=stress_name('depth_' + method, 64, 1000, lr, seed, tied),
                                    method=method, lr=lr, alignment=False,
                                    config=dict(width=64, depth=1000, tied=tied, microbatch=8,
                                                precision='mp', seed=seed)))
    return cells

def _run(name, seed, **overrides):
    name += '' if seed == 42 else f'-ms{seed}'
    return name, config(diagnostics=False, run_name_suffix='quiz2-' + name, model_seed=seed,
                       **overrides)

def lm_runs(seed=42):
    runs = []
    for lr in (.0015, .003, .006): # Q2: 1.2288B tokens, cosine decay to zero, 3 x 3 LR-WD grid
        for wd in (.1, .2, .4):
            runs.append(_run(f'q2-cos-lr{lr:g}-wd{wd:g}', seed, tokens=1_228_800_000,
                            lr_schedule='cos', learning_rate=lr, weight_decay=wd))
    for lr in (.006, .01, .015): # Q3: Hyperball (AdamH), 2.4576B tokens, no weight decay
        runs.append(_run(f'q3-adamh-tok2457M-lr{lr:g}', seed, tokens=2_457_600_000,
                        optimizer_name='adamh', learning_rate=lr, weight_decay=0.))
    for batch, lr, wd in ((8, .00075, .1), (256, .0015, 1.9439672364117835)): # Q4: 614.4M tokens
        for beta1 in (0., .9, .98):
            runs.append(_run(f'q4-b{batch}-beta1{beta1:g}', seed, tokens=614_400_000, batch=batch,
                            learning_rate=lr, weight_decay=wd, beta1=beta1))
    for lr in (.00075, .0015, .003, .006, .012): # Q5: almost-muP at width 1024, 153.6M tokens
        runs.append(_run(f'q5-almost_mup-w1024-lr{lr:g}', seed, tokens=153_600_000,
                        model_config=width_model_config(1024),
                        model_builder='experiments.quiz2.prescriptions:AlmostMuPLM',
                        optimizer_builder='experiments.quiz2.prescriptions:build_optimizer',
                        learning_rate=lr))
    for width in (128, 256, 512, 1024): # Q6: Hyperball (AdamH) across widths, 153.6M tokens
        model = {} if width == 512 else dict(model_config=width_model_config(width))
        for lr in (.006, .01, .015, .022, .03, .045):
            runs.append(_run(f'q6-adamh-w{width}-lr{lr:g}', seed, optimizer_name='adamh',
                            learning_rate=lr, **model))
    for label, (lr, wd) in Q7_RECIPES.items(): # Q7: batch 512, 614.4M tokens
        runs.append(_run(f'q7-b512-{label}', seed, tokens=614_400_000, batch=512,
                        learning_rate=lr, weight_decay=wd))
    return runs

```

Name _____

SUNet ID _____

your Stanford login, e.g. jdoe; not the 8-digit ID number

Check one box in every row and fill in all three rationale fields of each question. Only these two pages are graded.

Q1. Depth scaling with a shared block

20 points

(a) ρ 4 points per row	1. (0,10)	2. [10,100)	3. [100,1000)	4. [1000, ∞)
A. Standard μP	☐	☐	☐	☐
B. Depth- μP	☐	☐	☐	☐
C. CompleteP	☐	☐	☐	☐

(b) lowest loss 8 points ☐1. A. Standard μP ☐2. B. Depth- μP ☐3. C. CompleteP ☐4. All three within .005

Assignment problem cited	Observations from the experiment	Your rationale

Q2. Predicting loss under joint LR and WD changes

20 points

(a) ranking 10 points ☐1. $L_A < L_B < L_F < L_E < L_D < L_C$ ☐2. $L_D < L_B < L_F < L_E < L_A < L_C$
☐3. $L_F < L_E < L_A < L_B < L_C < L_D$ ☐4. $L_C < L_E < L_F < L_A < L_B < L_D$

(b) $L_C - 2.8354$ 10 points ☐1. [0,.005) ☐2. [.005,.015) ☐3. [.015,.025) ☐4. [.025, ∞)

Assignment problem cited	Observations from the experiment	Your rationale

Q3. Extrapolating Hyperball's optimal LR

20 points

fitted optimum at 2.4576B tokens 20 points ☐1. [.0075, ∞) ☐2. [.0055,.0075) ☐3. [.004,.0055) ☐4. (0,.004)

Assignment problem cited	Observations from the experiment	Your rationale

Name _____

SUNet ID _____

your Stanford login, e.g. jdoe; not the 8-digit ID number

Q4. Batch size and momentum

20 points

(a) ranking 5 points per row	1. $L_C < L_B < L_A$	2. $L_A < L_B < L_C$	3. $L_A < L_C < L_B$	4. $L_B < L_C < L_A$
$B = 8$	☐	☐	☐	☐
$B = 256$	☐	☐	☐	☐
(b) $L_A - L_B$ 5 points per row	1. $(-\infty, .01)$	2. $[.01, .06)$	3. $[.06, .15)$	4. $[.15, \infty)$
$B = 8$	☐	☐	☐	☐
$B = 256$	☐	☐	☐	☐

Assignment problem cited	Observations from the experiment	Your rationale
--------------------------	----------------------------------	----------------

Q5. Does learning-rate transfer survive changing the readout?

20 points

- (a) $\eta^*(1024)/\eta^*(512)$ 10 points ☐ 1. $(0, .5)$ ☐ 2. $[.5, .8)$ ☐ 3. $[.8, 1.25)$ ☐ 4. $[1.25, \infty)$
- (b) penalty of LR .003 10 points ☐ 1. $[.15, \infty)$ ☐ 2. $[.06, .15)$ ☐ 3. $[.01, .06)$ ☐ 4. $[0, .01)$

Assignment problem cited	Observations from the experiment	Your rationale
--------------------------	----------------------------------	----------------

Q6. Hyperball across widths

20 points

- (a) exponent p 10 points ☐ 1. $(-\infty, -.75)$ ☐ 2. $[-.75, -.25)$ ☐ 3. $[-.25, .25)$ ☐ 4. $[.25, \infty)$
- (b) penalty at width 1024 10 points ☐ 1. $[0, .005)$ ☐ 2. $[.005, .02)$ ☐ 3. $[.02, .08)$ ☐ 4. $[.08, \infty)$

Assignment problem cited	Observations from the experiment	Your rationale
--------------------------	----------------------------------	----------------

Q7. Scaling LR and WD to batch size 512

20 points

- (a) lowest-loss recipe 10 points ☐ A ☐ B ☐ C ☐ D ☐ E
- (b) $L_A - \min(L_A, \dots, L_E)$ 10 points ☐ 1. $[0, .02)$ ☐ 2. $[.02, .05)$ ☐ 3. $[.05, .15)$ ☐ 4. $[.15, \infty)$

Assignment problem cited	Observations from the experiment	Your rationale
--------------------------	----------------------------------	----------------